

SISTEMA DE GESTIÓN DE INCIDENCIAS PARA IPS (RIHO)

Asignatura: Ingeniería de software 1.

Profesor: Feibert Alirio Guzmán Pérez.

Estudiantes: Juan Esteban Ospina Zapata – Jeronimo
Mesa Carvajal

Cooperación Universitaria Lasallista

Ingeniería informática

29 de mayo de 2026

1. Entrega de requisitos (15%)

1.1 Contexto del proyecto RIHO:

El proyecto RIHO (Reporte de Incidencias Hospitalarias y Operativas) es un sistema web corporativo diseñado específicamente para la gestión, control y mitigación de fallas internas dentro de una Institución Prestadora de Servicios de Salud (IPS). El sistema centraliza el reporte de anomalías tecnológicas, operativas o de infraestructura, permitiendo un seguimiento riguroso y en tiempo real para asegurar que la prestación de los servicios de salud no se vea interrumpida.

1.2. Requisitos funcionales (RF)

- **RF-01: Registro de Incidencias:** El sistema debe permitir al personal asistencial y administrativo de la IPS reportar una incidencia de forma rápida, capturando el área afectada, la descripción del suceso y un nivel de prioridad inicial.
- **RF-02: Control de Estados:** El sistema debe procesar el ciclo de vida completo de cada caso (Estados: Pendiente, En Proceso, Resuelto, Mitigado) de manera dinámica mediante scripts especializados (`actualizar_estado.php`).
- **RF-03: Bitácora de Seguidimientos:** El sistema debe almacenar registros históricos cronológicos para cada reporte, permitiendo añadir observaciones y evidencias técnicas conforme avance la solución.
- **RF-04: Gestión de Roles y Permisos:** El software debe restringir las acciones basándose en perfiles de usuario (Operador, Técnico de Soporte, Administrador de la IPS).

1.3. Requisitos No Funcionales (RNF)

- **RNF-01: Seguridad y Confidencialidad de Datos:** Al operar en un entorno de salud, el sistema debe cumplir con estrictas políticas de privacidad, garantizando que el acceso a las incidencias esté restringido únicamente al personal autorizado.
- **RNF-02: Arquitectura y Rendimiento Transaccional:** El motor debe soportar accesos concurrentes e inserciones continuas de seguimientos sin generar bloqueos en las tablas del servidor de datos.
- **RNF-03: Interfaz Corporativa de Alta Usabilidad:** La interfaz de usuario debe implementar una estética moderna y limpia enfocada al sector Health-Tech, facilitando la lectura rápida y reduciendo la carga cognitiva de los usuarios en entornos hospitalarios de alta presión.

2. Diseño Arquitectónico (20% - junto a diseño de datos)

2.1. Diagrama y Estructura del Sistema

El sistema RIHO adopta un patrón de arquitectura Cliente-Servidor de Tres Capas, segmentando las responsabilidades de la siguiente manera:

- **Capa de Presentación (Frontend):** Construida con HTML5, CSS3 y JavaScript para ofrecer una experiencia de usuario interactiva, fluida y con un diseño corporativo limpio.
- **Capa de Lógica de Negocio (Backend):** Implementada en PHP, encargada de recibir las peticiones del cliente, aplicar las reglas lógicas del negocio (validación de formularios, control de sesiones) y procesar scripts críticos como `actualizar_estado.php`.
- **Capa de Datos (Persistencia):** Administrada en el motor relacional SQL Server, donde se ejecutan de manera segura

todas las transacciones, modificaciones e inserciones del sistema.

2.2. Justificación de Decisiones Tecnológicas

- **PHP:** Su facilidad para la modularización y el procesamiento ágil de scripts del lado del servidor lo convierten en la opción ideal para estructurar un flujo dinámico de reportes y seguimientos en tiempo real.
- **SQL Server:** En un entorno hospitalario corporativo, la robustez, consistencia y seguridad de la información son mandatorias. SQL Server provee un entorno transaccional altamente eficiente y seguro para el manejo de base de datos relacionales críticas.

3. DISEÑO DE DATOS (20% - junto a diseño arquitectónico)

3.1. Modelo Entidad-Relación (MER)

El almacenamiento y persistencia del sistema RIHO se estructuran bajo un modelo relacional normalizado en **SQL Server**, el cual asegura la integridad referencial y evita la duplicidad de la información. El flujo se compone de las siguientes entidades principales:

- **Usuarios (Usuarios):** Almacena el personal de la IPS (ID, nombre, correo, contraseña con hash y rol).
- **Reportes / Incidencias (Reportes):** Tabla central que registra cada evento (ID_Reporte, título, descripción, fecha_creación, ID_Usuario, ID_Estado).
- **Estados (Estados):** Catálogo maestro que define la fase actual del reporte (*Pendiente, En Proceso, Resuelto, Mitigado*).
- **Seguimientos (Seguimientos):** Bitácora relacional donde se registran los comentarios y avances técnicos de un reporte específico (ID_Seguimiento, ID_Reporte, comentario, fecha_registro, ID_Usuario).

3.2. Diccionario de Datos (Tablas Principales)

Tabla: Reportes

Campo	Tipo de Datos	Restricciones	Descripción
id_reporte	INT	PRIMARY KEY, IDENTITY	Identificador único y correlativo de la incidencia.
titulo	VARCHAR(150)	NOT NULL	Breve resumen o título del problema reportado.
descripcion	TEXT	NOT NULL	Detalle exhaustivo de la falla presentada en la IPS.
fecha_creacion	DATETIME	DEFAULT GETDATE()	Fecha y hora exacta en la que se registró el reporte.
id_usuario	INT	FOREIGN KEY (Usuarios)	Identificador del empleado de la IPS que genera la alerta.
id_estado	INT	FOREIGN KEY (Estados)	Estado actual en el ciclo de

Campo	Tipo de Datos	Restricciones	Descripción
			vida de la incidencia.

Tabla: Seguimientos

Campo	Tipo de Datos	Restricciones	Descripción
id_seguimiento	INT	PRIMARY KEY, IDENTITY	Identificador único de la bitácora de avance.
id_reporte	INT	FOREIGN KEY (Reportes)	Relación directa con el reporte que se está atendiendo.
comentario	TEXT	NOT NULL	Notas técnicas, avances o soluciones parciales aplicadas.
fecha_registro	DATETIME	DEFAULT GETDATE()	Momento exacto en el que el técnico añade el comentario.
id_usuario	INT	FOREIGN KEY (Usuarios)	Identificador del técnico o administrador que atiende el caso.

Tabla: Estados

Campo	Tipo de Datos	Restricciones	Descripción
id_estado	INT	PRIMARY KEY, IDENTITY	Código único para el estado del flujo de trabajo.
nombre_estado	VARCHAR(50)	NOT NULL	Nombre descriptivo (Pendiente, En Proceso, Resuelto, Mitigado).

Tabla: Usuarios

Campo	Tipo de Datos	Restricciones	Descripción
id_usuario	INT	PRIMARY KEY, IDENTITY	Identificador único de la cuenta de usuario.
nombre	VARCHAR(100)	NOT NULL	Nombre completo del colaborador de la IPS.
correo	VARCHAR(100)	NOT NULL, UNIQUE	Correo institucional utilizado para el acceso al sistema.
password	VARCHAR(255)	NOT NULL	Contraseña encriptada para

Campo	Tipo de Datos	Restricciones	Descripción
			seguridad de la cuenta.
rol	VARCHAR(30)	NOT NULL	Perfil asignado dentro del sistema (Admin, Técnico, Usuario).

3.3. Reglas de Integridad y Relaciones

- **Integridad Referencial:** No es posible eliminar un registro de la tabla Estados si existen reportes activos asociados a dicho estado.
- **Eliminación en Cascada Controlada:** Si un reporte es descartado o eliminado por administración, sus seguimientos asociados se eliminan en cascada (ON DELETE CASCADE) para evitar registros huérfanos en la base de datos de SQL Server.

4. IMPLEMENTACIÓN DEL CÓDIGO Y CONTROL DE VERSIONES (20%)

4.1. Estrategia en Git y Repositorio Remoto (GitHub)

Para la gestión del código fuente de **RIHO_2**, se utilizó **Git** como sistema de control de versiones distribuido, alojando el repositorio remoto en la plataforma **GitHub**. La estrategia de desarrollo se basó en el registro incremental de cambios mediante *commits* ordenados y descriptivos, lo que permite mantener una trazabilidad completa del proyecto. El archivo README.md principal en la raíz actúa como la *Landing Page* técnica del proyecto, donde se presenta el software a la comunidad y a los evaluadores.

4.2. Estructura Real de Directorios del Proyecto

El software se organizó de manera modular en la raíz del servidor local bajo las siguientes carpetas y componentes esenciales:

- `archivos_reportes / archivos_reportes_us / uploads`: Carpetas destinadas al almacenamiento y gestión segura de archivos adjuntos y evidencias multimedia de las incidencias reportadas en la IPS.
- `assets / datatables`: Módulos de librerías y estilos CSS/JS modernos encargados de renderizar la interfaz corporativa limpia de alto nivel (*Health-Tech*), garantizando tablas interactivas y fluidas para el análisis de datos.
- `vista`: Directorio que aísla los componentes de la interfaz de usuario para modularizar la presentación.
- **Scripts de Lógica y Procesamiento PHP (Raíz)**: Componentes clave encargados de resolver el ciclo de vida del reporte como `actualizar_estado.php`, `actualizar_asignacion.php`, `agregar_seguimiento.php`, `guardar_comentario.php` y los módulos de control de credenciales (`creden_procesar_tipo.php`, `creden_actualizar.php`, etc.).

4.3. Buenas Prácticas y Código Limpio

El desarrollo en **PHP** se estructuró bajo lineamientos de código limpio (*Clean Code*):

- **Principio de Responsabilidad Única (SOLID)**: Cada archivo ejecuta una función atómica dentro del flujo del sistema de la IPS. Por ejemplo, `inicio.php` gestiona la redirección del panel principal, mientras que la mutación de los estados lógicos de las incidencias en SQL Server es delegada exclusivamente a scripts especializados como `actualizar_estado.php`.
- **Prevención de Vulnerabilidades**: Se implementaron consultas preparadas y parametrizadas para la comunicación con SQL Server, mitigando riesgos de inyección SQL en formularios críticos como `login.php`.

5. PLAN DE PRUEBAS DEL SISTEMA

5.1. Estrategia de Pruebas Realizadas

Para asegurar la estabilidad de RIHO_2, se diseñó un plan de pruebas basado en dos niveles operativos:

- a) **Pruebas Unitarias:** Validación del comportamiento aislado de archivos críticos. Se comprobó que el script de autenticación (login.php) procese correctamente las credenciales y maneje las excepciones redireccionando a login_not_success.html o noseencontro.html en caso de errores de acceso.
- b) **Pruebas de Integración (Flujo Completo):** Se testeó el ciclo de vida de una incidencia simulando la operación diaria de la IPS:

[inicio_reporte.php] → [observar_reporte.php] → [actualizar_estado.php] → [agregar_seguimiento.php]

5.2. Escenario de Prueba de Integración Exitoso

- **Fase 1 (Apertura):** Mediante ini_inicio_reporte.php se registra un fallo de infraestructura en un área hospitalaria.
- **Fase 2 (Auditoría y Asignación):** A través de observar_reporte.php, el administrador visualiza el caso y ejecuta actualizar_asignacion.php para delegar el técnico encargado.
- **Fase 3 (Mitigación):** El técnico añade avances en tiempo real usando agregar_seguimiento.php y actualiza el ciclo del caso mediante actualizar_estado.php, impactando las tablas relacionales en SQL Server sin bloqueos transaccionales.

6. DESPLIEGUE LOCAL Y CONFIGURACIÓN DEL ENTORNO

6.1. Requisitos Previos del Sistema

- **Servidor Web Local:** Apache (mediante la suite XAMPP, apuntando al directorio raíz observable en la captura del entorno: C:\xampp\htdocs\RIHO_2).
- **Entorno Backend:** PHP 8.x con extensiones transaccionales para SQL Server configuradas (sqlsrv).
- **Persistencia:** Motor relacional **Microsoft SQL Server**.

6.2. Instrucciones de Despliegue

- a) Clonar el repositorio dentro del directorio local C:\xampp\htdocs\RIHO_2.
- b) Configurar los parámetros de red y credenciales de acceso del motor SQL Server en los archivos lógicos de conexión del backend.
- c) Iniciar los servicios en XAMPP y acceder localmente mediante el navegador web en la dirección http://localhost/RIHO_2/login.html.

7. DOCUMENTACIÓN FINAL Y REPOSITORIO DE GITHUB

7.1. Estructura del Repositorio Digital

El proyecto RIHO_2 se encuentra centralizado de forma pública en GitHub, estructurado de la siguiente manera:

- **README.md (Landing Page):** Documento principal que detalla de forma ejecutiva los alcances del sistema, la arquitectura de tres capas y las tecnologías empleadas.
- **RIHO_2_Final.pdf:** Este informe de ingeniería completo cargado en la raíz del repositorio para revisión docente.
- **Scripts y Código Fuente:** Estructura modular completa que incluye la raíz transaccional PHP mapeada y las carpetas de recursos del sistema.

7.2. Enlace a GITHUB:

[¡INGRESA AHORA A NUESTRO GITHUB!](#)